MODEL DEVELOPMENT / ARTIFACT 01

Algorithm types, connected.

10 algorithms and model families across tabular data, vision, language, and generative AI.

Yadunandan Mandalaneni | AIML-501 | Assignment 1.5

READ THE FRAMEWORK: learning style + domain + task

Learning style describes the training objective, not a permanent property of an architecture. Broad supervised/unsupervised categories appear alongside the specific training setup. In self-supervised training, targets come from the data itself. Conditional models and later fine-tuning can add supervision. Domain labels show selected common uses, not every possible application. All use cases are illustrative, not reported experiments.

01 Linear Regression [1]

Supervised

Tabular

Fits a weighted linear relationship between input features and a numeric target.

Use: Estimate a home price from area, location features, and property characteristics.

02 Random Forest [1]

Supervised

Tabular

Trains randomized decision trees on resampled data, then averages predictions or votes.

Use: Predict customer churn from account activity and service history.

03 Gradient Boosting [1]

Supervised

Tabular

Adds weak learners sequentially, each improving the current ensemble against a chosen loss.

Use: Rank loan applications by default risk using labeled historical outcomes.

04 Convolutional Neural Network [2]

Supervised: classifier setup

Computer Vision

Learns shared local filters and progressively combines visual features to predict a class.

Use: Flag visible manufacturing defects in labeled product photographs.

05 K-Means [1]

Unsupervised

Tabular / Computer Vision

Alternates assigning points to the closest centroid and updating each cluster mean.

Use: Group customers by scaled purchasing behavior without predefined segment labels.

06 Principal Component Analysis [1]

Unsupervised

Tabular / Computer Vision

Projects centered data onto orthogonal directions that retain the most variance.

Use: Reduce correlated sensor measurements to fewer features for visualization.

07 BERT [3]

Unsupervised/self-supervised pretraining; supervised fine-tuning

NLP

Uses bidirectional context to recover masked tokens; original BERT also used a sentence-pair objective.

Use: Fine-tune a pretrained encoder on labeled support tickets to classify intent.

08 GPT-style Transformer [4]

Unsupervised data / self-supervised pretraining

NLP / Generative AI

Uses causal attention to predict the next token from preceding tokens, then generates sequentially.

Use: Draft a product description from a supplied brief, followed by human review.

09 Generative Adversarial Network [5]

Unsupervised: unconditional setup

Computer Vision / Generative AI

A generator synthesizes samples while a discriminator learns to distinguish generated from real data.

Use: Generate synthetic product-image variations for creative exploration.

10 Diffusion Model [6]

Unsupervised data / self-supervised denoising

Computer Vision / Generative AI

Learns to reverse added noise; repeated denoising transforms a random sample into generated content.

Use: Generate image concepts; text-conditioned variants guide generation with prompts.

CHOOSE BY THE TASK

Predict a known outcome: supervised models. Discover groups: K-Means. Compress features: PCA. Understand text: BERT-based classification. Generate new content: GPT, GANs, or diffusion. Evaluate predictive models on held-out data; assess generated outputs for quality, diversity, and factual reliability.

Source key: [1] scikit-learn User Guide [2] He et al., 2015 [3] Devlin et al., 2019 [4] Radford et al., 2019 [5] Goodfellow et al., 2014 [6] Ho et al., 2020. Context: [7] Costa & Trojahan, 2024. Full references and selection notes accompany this infographic.

FRAMEWORK COMPANION / SELECTION AND RELATIONSHIPS

From concepts to model choices.

A model choice starts with the desired output and the information available for training.

DOMAIN CROSSWALK

Method	Tabular	Vision	NLP	GenAI
Linear regression	Yes	-	-	-
Random Forest	Yes	-	-	-
Gradient Boosting	Yes	-	-	-
CNN classifier	-	Yes	-	-
K-Means	Yes	Yes	-	-
PCA	Yes	Yes	-	-
BERT encoder	-	-	Yes	-
GPT-style model	-	-	Yes	Yes
GAN	-	Yes	-	Yes
Diffusion	-	Yes	-	Yes

Selected common applications; a dash does not mean impossible.

FOUR DIFFERENT QUESTIONS

Supervised

Training examples include a target supplied for the task, such as a defect label or sale price.

Unsupervised

The objective discovers structure or models a distribution without human-provided task labels.

Self-supervised

Targets are constructed from the input, such as masked tokens, next tokens, or injected noise. It is grouped here with learning from unlabeled data.

Generative

Describes the goal of producing samples or content. It does not specify one learning style or one architecture.

RELATIONSHIPS THAT CHANGE THE DECISION

Random Forest vs. Boosting

Both combine trees for labeled prediction. A forest averages randomized trees to reduce variance. Boosting builds learners sequentially to improve a loss. Compare both with the same validation split; neither is automatically best. [10]

K-Means vs. PCA

K-Means assigns observations to groups. PCA creates new continuous feature directions. PCA may precede clustering, but retained variance alone does not show that useful groups exist. [1, 11]

BERT vs. GPT

BERT uses bidirectional context to learn representations. A task head can classify a complete text. A GPT-style base model uses preceding context to predict and generate the next token. Both can be adapted after pretraining. [3, 4]

ILLUSTRATIVE SCENARIO / ONE RETAIL BUSINESS, FOUR TASKS

Business need	Candidate approach	Input, output, and quality check
Predict demand	Linear regression; forest; boosting	Past sales and known features -> predicted units. Compare MAE/RMSE on a later time period and against a simple demand baseline. [8, 9]
Segment customers	K-Means; optional PCA	Scaled purchasing features -> candidate groups. Check stability, separation, and business usefulness; groups are not established customer truths. [1, 11]
Route support tickets	BERT + classification head	Ticket text plus intent labels -> intent class. Use per-class precision/recall and macro-F1; inspect errors on similar or rare intents. [3, 8]
Create campaign content	GPT; GAN; diffusion	Brief -> draft text; noise or conditioning -> image candidates. Review prompt adherence, factual accuracy for text, image quality, and output diversity. [4-6]

A DEFENSIBLE SELECTION WORKFLOW

Define the output and error cost -> inspect labels and data quality -> choose a simple baseline -> split data to match intended use -> tune on training/validation data -> evaluate once on an untouched test set. Fit preprocessing using training data only. Time-dependent tasks need a split that respects time. [8, 9]

PORTFOLIO CONTEXT / TRACEABLE SOURCES

Purpose, process, and evidence.

A conceptual artifact for Model Development, supported by documentation and original research.

Introduction and description

A visual reference connects ten representative algorithms and model families across tabular data, computer vision, NLP, and generative AI. The companion guide adds relationships, selection examples, and evaluation considerations.

Objective

Classify the selected training setups, explain how each method works, and connect each one to a practical task. Help a reader distinguish an algorithm, an architecture, a domain, and a generation objective.

Process

Reviewed the assignment directions and written course resource; checked official documentation and original papers; organized the framework by training objective; revised ambiguous labels; added a domain matrix, comparisons, and a worked selection scenario; rendered and inspected the PDF.

Tools and technologies used

ChatGPT/Codex assisted research synthesis, drafting, and implementation. Python, ReportLab, and pypdf produced the PDF; HTML/CSS provide the accessible portfolio entry. The source is prepared for GitHub Pages.

Value proposition and unique value

The guide connects definitions with decisions. It makes self-supervised pretraining and supervised adaptation explicit, separates clustering from feature compression, and treats generation as a task with its own evaluation needs.

Relevance

Designed for learners and colleagues discussing early model choices. It supports clear technical communication and provides a foundation for later implementation projects. Applications are illustrative; no trained models or measured results are claimed.

REFERENCES

[1] scikit-learn developers. User Guide: supervised learning, clustering, and matrix decomposition. https://scikit-learn.org/stable/user_guide.html

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep Residual Learning for Image Recognition. https://arxiv.org/abs/1512.03385

[3] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. https://arxiv.org/abs/1810.04805

[4] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[5] Goodfellow, I., et al. (2014). Generative Adversarial Networks. https://arxiv.org/abs/1406.2661

[6] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. https://arxiv.org/abs/2006.11239

[7] Costa, B., & Trojahan, M. (2024, August 28). What are the differences between AI, ML, DL, and Gen AI? e-Core. https://www.e-core.com/na-en/articles/differences-between-ai-ml-dl-gen-ai

[8] scikit-learn developers. (n.d.). Metrics and scoring: Quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html

[9] scikit-learn developers. (n.d.). Cross-validation: Evaluating estimator performance. https://scikit-learn.org/stable/modules/cross_validation.html

[10] scikit-learn developers. (n.d.). Ensembles: Gradient boosting, random forests, bagging, voting, stacking. https://scikit-learn.org/stable/modules/ensemble.html

[11] scikit-learn developers. (n.d.). Decomposing signals in components: Principal component analysis. https://scikit-learn.org/stable/modules/decomposition.html#pca

Documentation accessed November 1, 2026. Examples are proposed applications, not claims about named commercial deployments. The written e-Core article provides AI/ML/DL/GenAI context; model mechanisms are supported by the technical sources above.